ENGINEERING STRATEGY · PREPARED FOR THE CTO · AUGUST 2026

Every harness can spend your money. Piko is the one that *answers for it.*

A coding-agent harness built around one metric — dollars per completed task — where spend limits are enforced by the runtime before the money leaves, every priced request is itemized in a per-run ledger, and the benchmark numbers are governed strictly enough to catch our own overclaims.

$26.24
measured spend for an 89-task official benchmark run — itemized per priced trial with stop reasons; the ledger exposes the 18 unpriced trials instead of counting them as zero

33/89
official-suite tasks solved under deliberately tight spend caps — a documented floor, decomposed failure by failure

815
tokens of default fixed prefix (prompt + built-in tool schemas) vs the multi-thousand-token startup of rich-context harnesses; CI fails any growth past the committed baseline

20/24
architecture decision records accepted (4 proposed) — the written spine behind the claims in this document

EXECUTIVE SUMMARY

The answer first

> Model rankings flip every quarter; your agent bill compounds every day. Piko is built for the second fact: it is the harness where a run's

> maximum cost is enforced by the runtime before the money is spent, and where every reported number survives being checked against the repository — because we check.

- **Spend is a contract.** Every turn can carry a hard dollar ceiling (in headless mode the turn is the run; session-scoped ceilings are ADR 0026, proposed). The harness reserves each request's worst-case cost *before* sending it and blocks the request that would break the cap — a native per-run USD ceiling with pre-dispatch reservation and durable exposure accounting, which we have not found shipped together in any peer harness. ADR 0020 · accepted
- **Every priced request is itemized.** Runs price requests at true cached-token rates from a versioned rate table, and the ledger exposes unpriced and unknown requests instead of treating them as zero. Our official-suite benchmark ships with that per-trial ledger: cost, stop reason, and pricing coverage for every task. Ask a competitor for the same artifact.
- **The numbers are governed hard enough to catch us.** Tuning data is firewalled from headline claims, missing trials are synthesized into denominators wherever run metadata allows it, and reviews are committed to the repo — including the fact-check that found the previous draft of *this document* violating our own rules. It was rewritten; both versions' review trails are in the repo.
- **Honestly pre-1.0.** A committed adversarial review scored maturity 2.6/5 (docs/reviews). Of the three major defects selected for ADRs 0022–0024, two are implemented and hardened; ADR 0022 — a reproduced filesystem race — is ratified and not yet implemented, and several other review findings also remain open. Piko is not ready for untrusted workloads, and says so in its own README.

1 · THE MARKET PROBLEM

AI coding is becoming a budget line nobody can forecast

Three concrete facts, each favoring the harness over the model:

Models churn	Costs compound

The frontier changed hands repeatedly in twelve months. A workflow welded to one model re-bets itself every release. The harness — the loop that plans, executes, verifies, and accounts — survives every swap.

Ten engineers experimenting becomes a thousand engineers operationalizing. Agent spend scales with adoption, and the bill arrives before the governance does — unless the governance ships inside the runtime.

Meters, no brakes

Peer harnesses tell you what you spent, after. We reviewed one current multi-model harness that meters per-agent cost carefully and bounds rounds and timeouts — but ships no dollar ceiling, while multiplying model calls per prompt by design.

SO WHAT

The first harness that can put an enforced number on "worst case, this run costs X" wins a conversation capability benchmarks can't enter: the one with your CFO.

2 · THE EVIDENCE

An itemized bill for every claim — led by the evidence tier

EXHIBIT 1 · EVIDENCE TIER · OFFICIAL SUITE, DECOMPOSED

On 89 never-seen official tasks under deliberately tight spend caps: 33 solved, every non-solve attributed, every priced dollar itemized

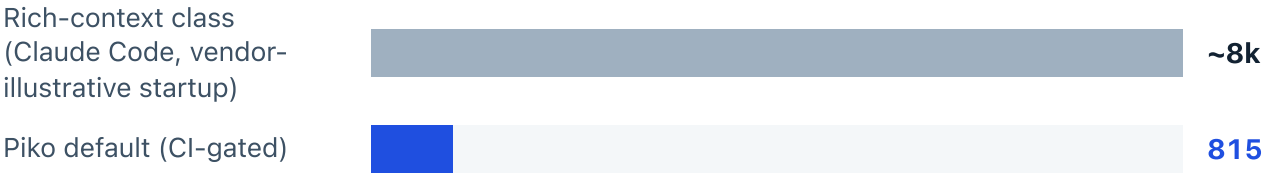
Solved · 33 · Scored failures · 11 · Stopped by our spend cap · 25 · Timeout policy · 7 · Test-rig infrastructure · 13

terminal-bench@2.0, 89 tasks piko had never seen or tuned on, n=1, every trial under piko's configurable per-trial spend cap, deliberately set tight. Measured spend: $26.24 across the 71 trials that produced a priced ledger (18 trials went unpriced, so total true spend is somewhat higher). 28 trials in all hit the ceiling: 25 unsolved, plus 3 that passed their verifiers *while* being cut off — evidence the cap was set inside the winning band and needs tuning upward, not removing. What the classes do NOT say: cap-stopped, timed-out, and infrastructure-failed trials reveal nothing about counterfactual capability,

so 37.1% is a documented floor, not a capability estimate. No baseline arm was run on this suite, and Terminus's public 78.0 (uncapped spend, harder 2.1 task set) is not comparable. Per-trial rewards, costs, and stop reasons: docs/benchmarks/2026-08-25-tb20/results.json, committed.

EXHIBIT 2 · STRUCTURAL EFFICIENCY

Piko's default fixed prefix is a fraction of the rich-context tier — and CI fails any growth past the committed baseline



Estimated tokens of default system prompt + built-in tool schemas per request. Project instructions, skills, and extensions add to piko's figure. The peer figure is Anthropic's own illustrative startup total, which its docs say varies; the internal prompt is unpublished, and most peers publish no comparable number at all.

The 815 is recomputed by CI on every build against a committed baseline (scripts/budget-baseline.json); any numeric growth past the baseline fails the build. The accompanying rule — growth must cite measured benefit — is procedural, enforced in review rather than parsed by CI. The rule has bitten: a plausible-sounding prompt line was deleted after three benchmark rounds showed its effect was statistical noise ($0.202 → $0.188 → $0.227 per failure), returning its tokens to the baseline. Cache-aligned prompt layout (ADR 0014) made 44% of input tokens bill at one-tenth price in the committed dev-suite run.

EXHIBIT 3 · DEVELOPMENT SIGNAL · TUNING TIER, LABELED AS SUCH

On the development suite, piko approaches the reference baseline's solve count at a lower measured cost per solve — a signal we deliberately do not headline

Cost per solved task · 10-task dev suite × 3 attempts · same model (GPT-5.5), same tasks and attempt counts · piko self-priced, baseline upper-bound estimate (its logs carry no cache split).

Why this is not the headline: these ten tasks are piko's development set — prompt fixes were tuned on their failures, and our governance bars tuning data from evidence claims (docs/benchmarks/2026-08-24-grid/rerun-and-heldout.md). It is shown here, labeled, because transparency beats omission: piko used fewer input tokens than the baseline on 8 of 10 tasks in this committed run (rerun-comparison.json). Metric note: $0.106 is total measured spend divided by solves; the artifact also carries a mean-cost-of-solved-trials field ($0.083) that excludes failure spend — both derive from the same ledger. A later same-suite run recorded 25/30 at $0.098 in the day's notes, but its per-trial artifacts were never committed before the run directory was lost — so we do not use those numbers,

and the repo now records that lesson. Held-out check on 10 seed-drawn unseen tasks: 12/27 valid trials with zero benchmark-gaming signatures found for the tuned failure class.

3 · THE GOVERNANCE SPINE

The complete decision ledger: 24 records, one root philosophy

Piko's claims aren't culture — they're numbered architecture decision records: 20 accepted, 4 proposed, never edited after acceptance, amended in public when reality disagrees. Every one traces to the same root: **cost**. Some make each request cheaper, some stop waste from compounding, some exist because incidents and integration churn are the most expensive tokens you'll ever buy.

Make every request cheaper

Value: *fixed context is a tax collected on every API call, forever, for every user — these decisions design the tax down and make it structurally unable to creep back.*

ADR	DECISION	WHAT IT GUARANTEES, IN PLAIN TERMS
0001	**Fixed-context budget, CI-enforced**	The 815-token default footprint is a build gate, not a goal — CI fails any numeric growth past the committed baseline; the cite-your-evidence rule is procedural, enforced in review.
0002	**Files and CLIs over MCP; five tools**	The smallest workable tool surface. Everything else is a file or a shell command — no protocol sprawl to pay for on every request.
0014	**Prompt-cache discipline**	Request layout keeps the prefix stable so provider caches hit — 44% of input tokens billed at one-tenth price in the committed development-suite run.
0017 · proposed	**$/completed task is the fitness function**	The root metric, written and awaiting ratification: every change is judged by one number, and prompt text pays "rent" in measured benefit or gets deleted — a rule already applied once against our own feature.

Stop runaway spend before it compounds

Value: *our benchmark forensics show failures, not solves, dominate agent bills — a wandering agent burned 3× a focused one on the same task. These decisions bound the expensive failure modes inside the loop, where the model can't negotiate.*

ADR	DECISION	WHAT IT GUARANTEES, IN PLAIN TERMS
0003	**Observable compaction**	Context growth — the quiet cost multiplier — is cut back into new, lineage-linked files, in the open, never silently rewritten.
0005	**Loop-side flail guard**	An agent repeating failing calls is stopped by the loop counting them — not by hoping the prompt persuades it to give up. (Scope is failure loops; overall spend is bounded by 0009/0020.)
0009	**Hard run budgets in the loop**	Turn, token, and time ceilings enforced where they can't be talked around.
0020	**Dollar accounting and spend ceilings**	The configurable spend cap, enforced per user turn by reserving each request's worst-case cost before dispatch — worst-case turn spend becomes a contract, not a distribution. Session-scoped and child-tree ceilings are ADR 0026, proposed.

Never pay for the same failure twice

Value: *a crash that loses history, a duplicated side effect, or a "did it run?" mystery costs engineer-hours — the most expensive line on any AI bill. These decisions make recovery cheap and honest — the journal reports what is unknown instead of guessing.*

ADR	DECISION	WHAT IT GUARANTEES, IN PLAIN TERMS
0007	**Write-ahead lifecycle journal**	Every action is journaled before it runs. After a crash the record says "outcome unknown" — it never guesses. It reports honestly rather than deduplicating; the record itself says idempotency needs tool-specific keys.
0008	**Strict provider contract**	A malformed model response becomes a typed failure, never a silently accepted answer you pay to debug later.
0011	**Persistent approve / edit / reject**	A gated action suspends the run into a saved state. A human — or a script — approves, edits, or rejects later. No spend and no risk taken because a session timed out.
0015	**Durable single-writer sessions**	One process writes a session's history; files are owner-only; corruption fails loudly instead of being skipped over.
0023	**Lock-capability session API**	Writing to a session without holding its lock is impossible — rejected by the type system and again at runtime.
0024	**Explicit stale-lock recovery**	After a crash, the harness fails loudly and names the recovery command — it never silently resumes older history or loses your newest work.

Make trust cheap enough to scale

Value: *one leaked credential or one escaped write can cost more than every token the harness will ever save — containment failures are the most expensive spend of all. These decisions shrink the blast radius, and the ledger states exactly where the boundary still leaks (0022, accepted, not yet implemented).*

ADR	DECISION	WHAT IT GUARANTEES, IN PLAIN TERMS
0004	**Sub-agents are headless self-spawns**	A sub-agent is just another piko process — one execution path to audit, not two. (The record notes the caveat: enabling delegation today rides on host-bash opt-in.)
0006	**Workspace containment; host bash deny-by-default**	Path-based containment plus deny-by-default host bash reduce exposure; the known parent-symlink race remains until ADR 0022 lands. Touching the host shell is an explicit operator opt-in.
0012	**Extensions are trusted controller code**	Plugins are explicitly trusted code — the design never pretends a plugin boundary is a security boundary.
0013	**Three separated event surfaces**	User output, session record, and telemetry are distinct channels; telemetry redacts by default.
0016	**Credential handling**	Child shells get a scrubbed environment; credentials travel by *name* in observability, never by value — tested with redaction turned off.
0018 · proposed	**Container sandbox executor**	The designed path to OS-grade isolation, behind a clean seam so it can land without rewiring the harness.
0022 · accepted, not implemented	**Descriptor-anchored containment**	Will close the filesystem race an adversarial review proved, with the reviewer's own exploit as the acceptance test. The decision is ratified; the code has not landed — this is the top engineering ask below.

Integrate once, keep it priced

Value: *integration churn is a recurring cost paid by every team that builds on you — versioned contracts convert it into a one-time cost.*

ADR	DECISION	WHAT IT GUARANTEES, IN PLAIN TERMS
0010	**Fail-closed automation contract**	Headless runs emit typed, versioned JSON and stable exit codes. Pipelines integrate without screen-scraping; changing an exit code requires a written amendment.
0019 · proposed	**Release and compatibility contract**	What a version number will promise — written down before anything ships, not after.
0021 · proposed	**Artifact data lifecycle**	Numbers cited anywhere must have their artifacts committed in the same change, or they are narrative, not evidence — a rule this project already paid to learn when a benchmark run's ledger was lost before commit.

> **SO WHAT**
>
> Ask any harness vendor two questions: "show me the written decision behind that claim" and "show me the time you enforced it against yourselves." Piko answers both with commit links — including the commit where this document's own first draft was corrected.

4 · COMPETITIVE POSITION

Peers optimize capability; piko's distinct bet is enforced economics

HARNESS	THESIS	FIXED CONTEXT	COST POSTURE	MATURITY
Claude Code	Rich context, subagents; quality first	~8k illustrative startup (varies; prompt unpublished)	Usage reporting; org-level budgets available via gateway tooling	**Production**
Codex CLI	Approval protocols, policy engine	not published	Usage reporting	**Production**
Terminus-2	Minimal research baseline	not published	Prices surfaced; turn limits; no dollar ceiling	**Research**

DeepSeek harness	Plugin-based harness; economics driven by low model prices	n/a	Low token prices; no per-run dollar ceiling documented	**Emerging**
Fusion-class	Run 2–5 models per prompt, merge	n/a	Round caps, timeouts, writer lease; cost displayed per agent; no dollar ceiling	**Experimental**
Piko	**Enforced cost per outcome**	**815 default, gated**	**Native per-run USD ceiling · pre-dispatch reservation · durable exposure accounting · self-pricing**	**Pre-1.0, hardening**

SO WHAT

The narrow claim that survives scrutiny: among these harnesses, piko is the only one we have found that ships a native per-run dollar ceiling with conservative pre-dispatch reservation and durable exposure accounting, in the runtime itself. Peers meter well and some can budget at the platform layer; none makes a single run's worst-case spend a runtime-enforced contract. And because piko composes with supported Anthropic and OpenAI-compatible models — a set that includes much of what competes for the frontier — cheaper tokens under its governance simply mean a better contract.

5 · WHY THESE NUMBERS CAN BE TRUSTED

Our bar charts are hard to fake — this document is the proof

The previous draft of this overview was fact-checked against the repository by an external model reviewer. It found the draft violating our own rules — headlining tuning data, citing an uncommitted review, rounding claims past their evidence. The review and the rewrite are both committed. That loop is the product's quality system, applied to its own marketing.

Tuning is firewalled from evidence	**Denominators are protected**

Tasks we optimized against are formally labeled tuning data and barred from headline claims — the rule that demoted this document's original headline. Evidence comes from seed-drawn unseen tasks and the official suite; on the held-out set, a scan of every failed trial for the tuned failure signature found zero gaming.

When a run's metadata exists, missing trials are synthesized into the score as explicit failures; when it doesn't, the tooling warns loudly that it cannot check. The one documented exclusion (a task no runtime-download installer can boot) is recorded as a limitation in the manifest, not silently dropped.

Reviews are committed, on the record

docs/reviews holds the adversarial review (2.6/5 scorecard, 16 findings), a recorded summary of the six-defect re-review of our first fix (its code fixes are commit bc34217), and the fact-checks of this document. Reproductions from the closed findings are permanent regression tests; ADR 0022's will land with its fix.

Corrections are dated commits

A flail-threshold claim was formally retracted, and cost accounting was separately corrected for cache pricing; the official-suite score was corrected upward (30→33) the same day a counting flaw surfaced; a lost run's numbers were demoted to narrative when their artifacts proved uncommitted. All in the repo with dates and diffs.

6 · WHAT WE NEED TO WIN

Three asks, sequenced by return per dollar

01 **Implement ADR 0022 — engineering time, $0**

The descriptor-anchored filesystem boundary is ratified but not built; it closes the last confirmed major security finding, with the reviewer's exploit as the acceptance test. Until it lands, piko's own README correctly bars untrusted workloads — landing it is the credibility unlock for any external evaluation.

02 **Map the cost/quality frontier — an estimated $15–30 of API credit**

Rerun the cap-stopped official tasks at progressively higher cap settings. This adds higher-budget points for the cap-stopped subset — the start of a real cost/quality curve — and locates where a dollar stops buying solves for those tasks — the number that becomes our default spend setting, and a chart built from exactly the per-trial ledgers piko already produces.

03 **Decide the license and sequence the ADR 0019 release checklist — a decision, $0**

The license is the first of several release-contract items (public packaging, provenance, support matrix, clean-machine install). Deciding it unblocks the sequence; the benchmark provenance and review trail become the launch story when the checklist is done — not before.

THE BOTTOM LINE

Capability is table stakes that reprices every quarter. The durable position is the harness that delivers outcomes at a bounded, itemized, auditable price. Piko already runs official benchmarks under enforced dollar ceilings and publishes the full bill — and its governance is strict enough that when our own marketing overreached, our own review process caught it and this document was rewritten. That loop is the asset.

Provenance. Every figure traces to committed artifacts in the piko repository: self-priced benchmark runs and per-trial ledgers (docs/benchmarks/2026-08-24-grid, docs/benchmarks/2026-08-25-tb20), the 24-record ADR index (docs/adr, 20 accepted / 4 proposed), and the committed reviews — the adversarial review with its 2.6/5 scorecard, its re-review, and the fact-check that drove this document's rewrite (docs/reviews). Dev-suite figures are tuning-tier by the repo's own rules and are labeled as such above. The Terminus public anchor (78.0, Terminal-Bench 2.1) is uncapped spend on a harder task set and is not comparable to piko's cost-capped 2.0 run. Peer comparisons reflect public documentation as of August 2026; where a peer's figure is not published, this document says so. Piko is pre-1.0 and unsuitable for untrusted workloads until ADR 0022 lands.